

Sprint de finalisation et préparation playtest — Activités 7 et 9

Ce document regroupe les preuves attendues pour les activités d'amélioration et de finalisation : état des lieux, priorisation MoSCoW, corrections critiques, stabilité, UI/UX et préparation de démonstration.

1. État des lieux fonctionnel

Fonctionnalité	Statut	Commentaire
Inscription / connexion JWT	✓ Terminé	Formulaire front + routes back
Hachage mot de passe	✓ Terminé	bcrypt
Rôles joueur/admin	✓ Terminé	player, admin, route /api/users admin
Partie jouable	✓ Terminé	démarrage, chat, actions, sauvegarde
Narration IA	✓ Code terminé / ⚠ billing à activer	OpenAI réel + fallback local
Combat	✓ Terminé	tour par tour, attaque, défense, fuite
Inventaire / loot	✓ Terminé	génération procédurale
Carte / déplacements	✓ Terminé	zones accessibles
Tests backend	✓ Terminé	21 tests
Tests frontend	✓ Terminé	13 tests
Déploiement VPS	✓ Terminé	http://89.116.111.166:8081/

2. Priorisation MoSCoW

Priorité	Tâche	Statut	Justification
Must	Authentification JWT complète	✓ Fait	Critère noté 10 points, sécurité obligatoire
Must	Routes API REST protégées	✓ Fait	Critère activité 3/5
Must	Frontend fonctionnel end-to-end	✓ Fait	Critère activité 4/6
Must	MCD/MLD + doc cadrage	✓ Fait	Livrables obligatoires
Should	Tests backend/front	✓ Fait	Non exigés par cadrage, mais sécurisent la note
Should	Déploiement VPS	✓ Fait	Non exigé, bonus de démonstration
Could	Rôles admin détaillés	✓ Partiel	Route admin existante, UI admin non prioritaire
Could	Refresh token	🕒 Backlog	Amélioration post-module
Won't	Multijoueur temps réel	✗ Hors périmètre	Non nécessaire pour le module

3. Corrections critiques réalisées

Correction	Risque initial	Résultat
Remplacement de X-User-Id par JWT	Usurpation d'identité	Auth stateless sécurisée
Ajout bcrypt	Mot de passe en clair possible	Hash \$2b\$. . . en base
Protection routes jeu	Accès libre à l'état de partie	401 sans token
Ajout bouton logout	Jeton persistant sans contrôle utilisateur	Déconnexion propre

Correction	Risque initial	Résultat
Fallback IA clarifié	Message trompeur si OpenAI billing KO	Message explicite
Mise à jour E2E	Parcours obsolète après auth	Parcours inscription → jeu

4. Stabilité et tests

Type de test	Commande	Résultat attendu
Backend	<code>pytest -q</code>	21 passed
Frontend unitaires	<code>cd frontend; npm test</code>	13 passed
Build frontend	<code>cd frontend; npm run build</code>	build OK
Healthcheck VPS	<code>GET /api/health</code>	200 OK
Auth prod	register puis state avec token	201 puis 200
Auth prod sans token	<code>GET /api/state</code> sans header	401

5. Préparation playtest

Scénario de playtest principal

1. Ouvrir `http://89.116.111.166:8081/`.
2. Créer un compte joueur.
3. Se connecter automatiquement après inscription.
4. Démarrer la partie.
5. Envoyer une action libre au MJ.
6. Lancer un jet 2D6.
7. Fouiller une zone pour générer du butin.
8. Déclencher une rencontre.
9. Faire une action de combat.
10. Sauvegarder puis se déconnecter.

Objectifs observables

Objectif	Critère de succès
Authentification	Compte créé et token stocké côté navigateur
Communication front/back	Actions visibles immédiatement dans l'UI
IA	Réponse OpenAI si billing actif, fallback local sinon
Stabilité	Aucun crash pendant le parcours
UX	L'utilisateur comprend les boutons principaux

6. Script de démo 5 minutes

1. **Concept** — jeu RPG textuel fullstack avec MJ IA.
2. **Architecture** — React ↔ FastAPI ↔ SQLite/OpenAI.
3. **Sécurité** — montrer `register/login`, JWT, route protégée sans token.
4. **Flux de données** — action frontend → API → état mis à jour.
5. **IA** — narration streamée en SSE.
6. **Tests / qualité** — citer `21 passed`, `13 passed`, build OK.
7. **Limites honnêtes** — billing OpenAI à activer, fallback local, refresh token en backlog.

7. Support de présentation conseillé

Slides recommandées :

1. Titre + objectif du jeu.
2. Architecture globale.
3. Modèle de données MCD/MLD.
4. API REST + sécurité JWT.
5. Démo du parcours utilisateur.
6. IA et fallback.
7. Tests / déploiement.
8. Analyse critique et feuille de route.

8. Rétrospective agile (Start / Stop / Continue)

Catégorie	Élément
Continue	Découpage en couches, commits atomiques, tests avant chaque incrément
Continue	Repli local systématique pour ne jamais casser la démo
Start	Restreindre le CORS et exiger un <code>JWT_SECRET</code> fort en prod
Start	Persister l'état de partie en base plutôt qu'en YAML
Stop	Laisser grossir <code>combat_action</code> / <code>streamSSE</code> sans découpage
Stop	Dépendre d'un unique compte OpenAI sans plan de facturation validé

9. Suivi d'avancement du sprint

Lot	Tâches	Terminées	Reste
Sécurité (JWT/bcrypt/rôles)	6	6	—
Frontend auth + gating	4	4	—
Robustesse IA / fallback	3	3	—
Tests & build	3	3	—
Déploiement VPS	2	2	—
Documentation de rendu	8	8	—
Total	26	26	0 bloquant

Le seul point ouvert n'est pas un blocage de code mais une action externe : **activer la facturation OpenAI** (ou fournir une clé liée à un compte actif) pour obtenir 100 % de la fonctionnalité IA distante en démonstration.

10. Definition of Done (critères de clôture)

Une fonctionnalité est considérée **terminée** si :

- ☑ le code est écrit et intégré sur `main` ;
- ☑ les tests associés passent (back et/ou front) ;

- ☒ le build frontend reste vert ;
- ☒ la fonctionnalité est vérifiable dans le parcours de démo ;
- ☒ la documentation correspondante est à jour ;
- ☒ aucun secret n'est commité dans le dépôt.